

Functional tests

sábado, 6 de setembro de 2025

10:27

Perfect 👍 Let's document what we did step by step so you can keep this as notes.

Notes: JSON Dumping & Post-Test Processing in Pytest

1. Problem

You wanted to:

- Save the **API responses** (JSON) from your functional tests.
- Store them in a **structured folder next to the test files**.
- Keep only the **last 3 results per test** to avoid folder growth.
- Automate this dumping process **after each test run**.

2. Utility Function (dump_json_result)

We created a **helper** to save JSON responses with timestamps and clean up old files.

```
import json
import os
from pathlib import Path
from datetime import datetime

def ensure_dir(path: Path):
    path.mkdir(parents=True, exist_ok=True)

def dump_json_result(
    data,
    test_name: str,
    target_folder: str | Path,
    filename_suffix: str | None = None,
    pretty: bool = True,
    keep_last: int = 3
) -> Path:
    base_folder = Path(target_folder).expanduser().resolve()
    ensure_dir(base_folder)
    ts = datetime.utcnow().strftime("%Y%m%dT%H%M%S")
    pid = os.getpid()
    suffix = f"_{filename_suffix}" if filename_suffix else ""
    filename = f"{test_name}_{ts}_{pid}{suffix}.json"
    path = base_folder / filename
    # Save JSON
    with open(path, "w", encoding="utf-8") as f:
        if pretty:
            json.dump(data, f, indent=2, ensure_ascii=False)
        else:
            json.dump(data, f, ensure_ascii=False)
    # Cleanup old files (keep only last N)
    all_files = sorted(
        base_folder.glob(f"{test_name}_*.json"),
        key=lambda p: p.stat().st_mtime,
        reverse=True
    )
    for old_file in all_files[keep_last:]:
        try:
```

```

        old_file.unlink()
        print(f"[dump_json_result] deleted old dump: {old_file}")
    except Exception as e:
        print(f"[dump_json_result] failed to delete {old_file}: {e}")
return path

```

✅ Features:

- Saves responses with unique filenames (testName_timestamp_pid.json).
- Pretty-prints JSON for readability.
- Automatically deletes older dumps beyond the last 3.

3. Automatic Fixture (auto_dump_after_test)

We added a **pytest autouse fixture** that runs after every test.

It checks if `request.node.last_response` was set and automatically saves it.

```

import pytest
from pathlib import Path
from test_api.ProcessSourceFiles.util import dump_json_result
@pytest.fixture(autouse=True)
def auto_dump_after_test(request):
    """
    Autouse fixture: after every test, if `request.node.last_response` is set,
    dump it as JSON into <same_folder_as_test>/api_test_results/.
    """
    yield
    last = getattr(request.node, "last_response", None)
    if last is not None:
        test_file = Path(str(request.node.fspath)) # the test file path
        output_dir = test_file.parent / "api_test_results"
        test_name = request.node.name
    try:
        path = dump_json_result(
            last,
            test_name=test_name,
            target_folder=output_dir
        )
        print(f"[auto_dump_after_test] dumped JSON for {test_name} -> {path}")
    except Exception as exc:
        print(f"[auto_dump_after_test] failed to dump JSON for {test_name}: {exc}")

```

4. Usage in Tests

In each test, after calling the API and parsing the JSON, we assign:

```
request.node.last_response = data
```

This makes the JSON available to the fixture for automatic dumping.

Example:

```

def test_v1_listFilesInUrl_SimpleMainFile(request):
    input_json = {"url": "https://github.com/.../SimpleMainFile"}
    response = requests.post(f"{BASE_URL}/api/v1/listFilesInUrl", json=input_json)
    data = response.json()
    # Register so fixture can dump automatically
    request.node.last_response = data
    assert response.status_code == 200
    assert isinstance(data, list)
    assert len(data) > 0

```

5. Resulting Folder Structure

For a test file `test_api/test_something.py`, results go into:

test_api/
test_something.py
api_test_results/
test_v1_listFilesInUrl_SimpleMainFile_20250906T123456Z_1234.json
test_v1_listFilesInUrl_SimpleMainFile_20250906T124500Z_1235.json
test_v1_listFilesInUrl_SimpleMainFile_20250906T125600Z_1236.json

- Each test name gets its own dumps.
- Old files (beyond 3) are deleted automatically.

6. Benefits

-  No manual dumping needed → runs automatically after every test.
-  Results stored **next to test files** → easy to find.
-  Limited to **last 3 files per test** → prevents clutter.
-  Useful for debugging API changes and regression testing.

👉 Next Step: if you want, we can add a **global flag** (e.g. env variable SAVE_TEST_OUTPUT=true) so dumps only happen when explicitly enabled.
Do you want me to extend it with that flag?